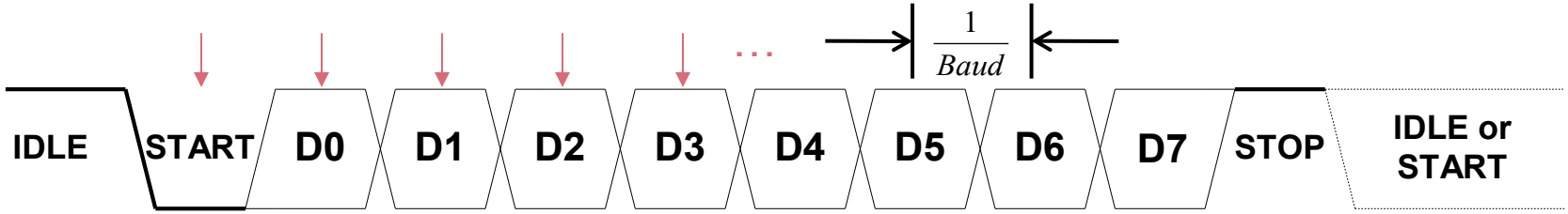
Exercise 12: UART Receiver

- We will implement the UART receiver to work with our transmitter
  - 8N1 frame format
  - 9600 Baud rate
- UART receiver monitors for falling edge on the line → this is the start bit
- Once this is detected, we want to wait half a Baud period before sampling the value on the line (sample in the middle of the period)
- From then on, we are sampling every Baud period for subsequent bits



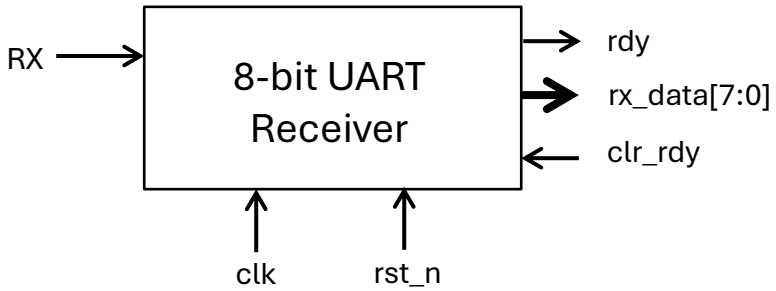
- Use a 9-bit shift register for this design.
  - Refer back to the UART TX design if you forget why 9 bits and why shift register.
- It should shift a total of 10 times, and the start bit should "fall off" the end of the 9-bit shift register. Bits [7:0] of the shift register are the received data.

Exercise 12: UART Receiver

- You are designing the datapath and FSM for this UART receiver
  - The UART receiver will monitor the **RX** line and assemble **rx_data**
  - **rdy** should be asserted once the byte is ready for consumer to read
  - **clr_rdy** should be asserted once the consumer has read the data to clear the **rdy** signal. This way they know the next rdy=1 is a new byte.

UART is an async serial protocol.
What does this mean about RX?
It is an async input!

What do we need to do to async inputs if we want to use it in our system?



Signal	Dir	Description
clk,rst_n	in	50MHz system clock & active low reset
RX	in	Serial data input
clr_rdy	In	Knocks down **rdy** when asserted
rx_data[7:0]	out	Byte received
rdy	out	Asserted when byte received. Stays high till start bit of next byte starts, or until **clr_rdy** asserted.

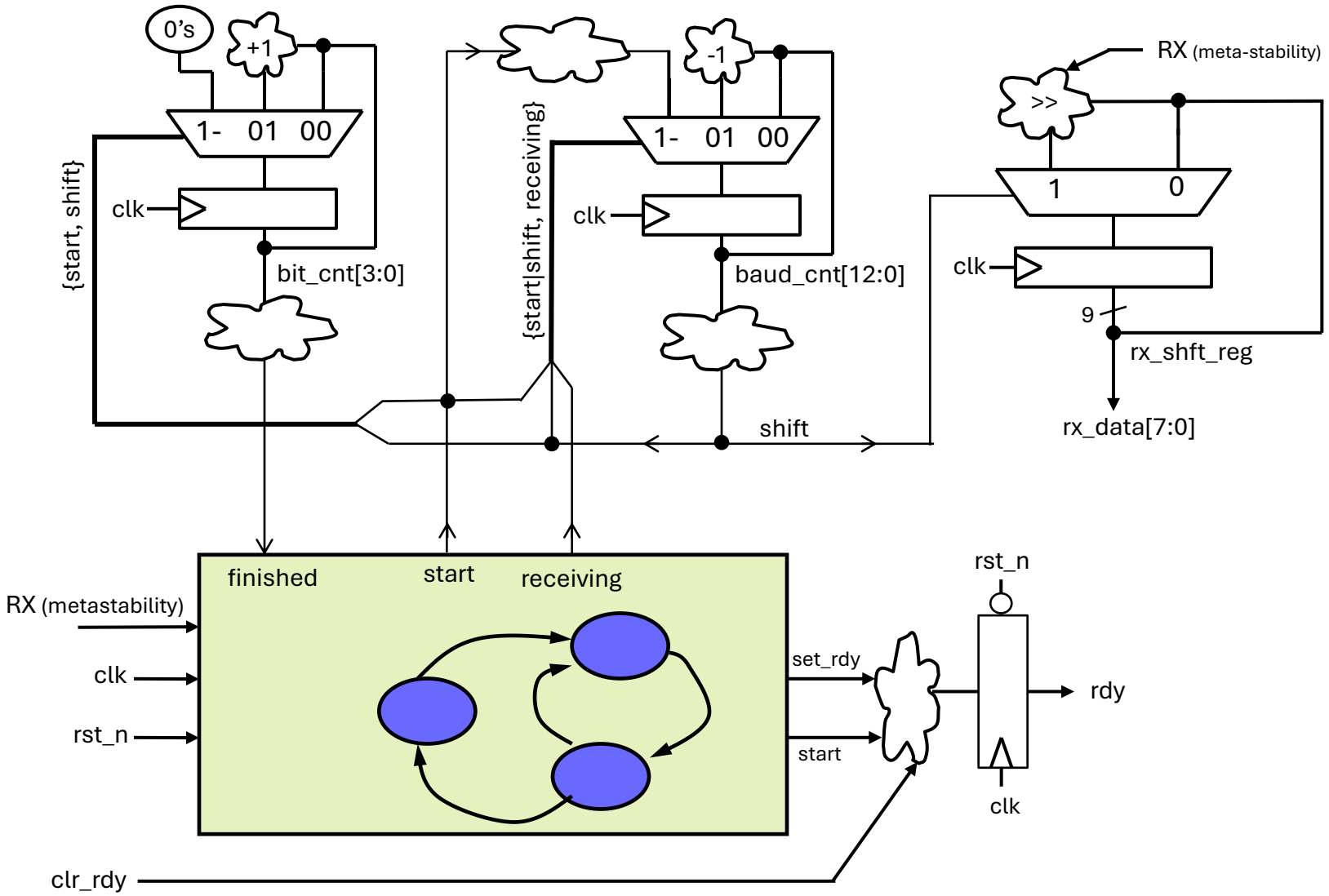
UART RX Design

- Follow the same process you use for designing the transmitter

- You will need:
  - Shift registers to hold data received
  - Counter for baud rate – sets up when to sample data
  - Counter for which bit we are at
  - FSM to manage control signals – when to do what
  - Synchronizer to handle metastability issue for RX

- Many components needed are similar to ones in TX. You can take your TX design and modify parts of it.

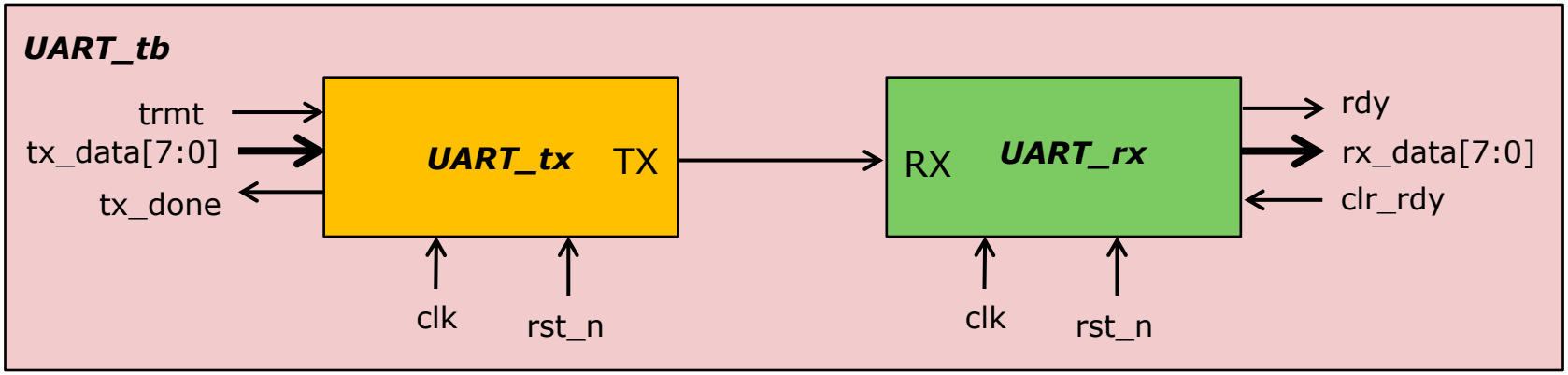
Possible Topology of UART_rx



Exercise 12: UART Receiver

- Your design should be called UART_rx
  - Implement the datapath and FSM
    - What is the minimum number of states you need?
- You should follow the FSM coding style introduced in the lecture videos

- Since you have a transmitter and a receiver, it is now easy to make a self-checking testbench. Architect the testbench as shown.
  - Does the 8-bit value you transmit match the value you receive when the transmission completes?
  - Also test **tx_done**, **rdy**, and **clr_rdy** functionality.



Submit: UART_rx.sv, UART_tb.sv, proof the testbench passed